



CREACIÓN DE COMPONENTES

CFGS Desarrollo de Aplicaciones Multiplataforma

Desarrollo de interfaces

Fernando Francisco Toro Rueda

Carmen Campos Fernández

CONTENIDO

1.	¿Qué es un componente reusable?	3
2.	tipos de componentes en wpf	3
3.	Creación de componentes UserControl	3
3.1.	LabeledInput.xaml	4
3.2.	LabeledInput.xaml.cs	6
3.3.	Uso del componente creado en MainWindow.xaml	8
3.4.	Creacion componente Custom Control	8
3.5.	Distribucion de componentes	11
	Paso 4: Mapeo de XMLNS	13
	Paso 5: Generar la DLL	13

1. ¿QUÉ ES UN COMPONENTE REUSABLE?

Un componente reusable es una unidad de interfaz gráfica (UI) que:

- encapsula diseño, comportamiento y datos.
- puede reutilizarse en diferentes ventanas o proyectos.
- se comporta como un control más de WPF (Button, TextBox, etc.).

2. TIPOS DE COMPONENTES EN WPF

Tipo	Qué es	Cuándo usarlo	Ejemplo
UserControl	Control compuesto a partir de otros controles	Cuando quieres unir varios controles existentes (Label + TextBox, por ejemplo)	Un campo con etiqueta: Label + TextBox
Custom Control	Control nuevo que hereda de Control o similar y usa plantillas (Generic.xaml→se guarda el diseño del control)	Cuando necesitas que sea temeable (cambiable con estilos) o más optimizado; que se pueda cambiar su apariencia de forma radical y sencilla.	Botón con icono personalizable
Behavior / Attached Property	Una clase que agrega funcionalidad extra a un control existente, sin modificarlo	Cuando quieres que un TextBox haga algo nuevo (por ejemplo, ejecutar un comando al pulsar Enter)	TextBoxBehaviors.EnterCommand
DataTemplate	Plantilla visual de un tipo de datos (no control en sí)	Cuando solo quieres cambiar cómo se muestra un tipo de datos en un ListBox, etc.	Mostrar una lista de personas con foto y nombre

3. CREACIÓN DE COMPONENTES USERCONTROL

En este apartado vamos a crear un componente paso a paso de tipo UserControl. Concretamente un *campo con etiqueta* (Label+TextBox):

- Selecciona Crear un nuevo proyecto.
- Busca "WPF Application (.NET Framework)".

3. Nómbrala, **DemoControles**
4. Ya en la pantalla del proyecto, si no vemos el **explorador de soluciones**, vamos al menú principal, pinchamos en **ver** y en **explorador de soluciones**.
5. Sobre el nombre **DemoControles**, botón derecho, Agregar --> Control de Usuario(WPF)/UserControl(WPF) --> lo nombramos **LabeledInput.xaml**
6. Se crean los siguientes ficheros correspondientes.

3.1. LABELEDINPUT.XAML

Copiamos el siguiente contenido en LabeledInput.xaml:

```
<UserControl x:Class=" DemoControles.Controls.LabeledInput "
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    x:Name="root">
    <StackPanel>
        <TextBlock Text="{Binding LabelText, ElementName=root}"
            FontWeight="Bold"/>
        <TextBox Text="{Binding Text, ElementName=root, Mode=TwoWay}"/>
    </StackPanel>
</UserControl>
```

Desglose del código:

1. <UserControl x:Class=" DemoControles.Controls.LabeledInput " ...>

- **<UserControl>**: Declara que este archivo define un control de usuario. Un UserControl es una forma de encapsular un grupo de elementos UI (User Interface) y su lógica asociada en un solo componente reutilizable.
- **x:Class=" DemoControles.Controls.LabeledInput "**: Especifica el nombre completo de la clase de C# (o VB.NET) que está asociada a este archivo XAML. Esta clase contendrá la lógica de "code-behind" para el control.
- **xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"**: Declara el espacio de nombres XML predeterminado para elementos WPF. Esto significa que elementos como UserControl, StackPanel, TextBlock, TextBox son reconocidos como parte de la biblioteca estándar de WPF.
- **xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"**: Declara el espacio de nombres XML para las características del lenguaje XAML. Esto permite usar prefijos como x:Class o x:Name.
- **x:Name="root"**: Asigna el nombre "root" a este UserControl. Esto es útil para referenciar el control desde dentro de su propio XAML o desde su code-behind.

2. <StackPanel>

- **<StackPanel>**: Es un panel de diseño que organiza sus elementos hijos en una sola línea, ya sea vertical u horizontalmente. Por defecto, es vertical. En este caso, contendrá una etiqueta (TextBlock) y una caja de texto (TextBox).

3. <TextBlock Text="{Binding LabelText, ElementName=root}" FontWeight="Bold"/>

- **<TextBlock>**: Es un control que muestra texto de solo lectura. Se usa comúnmente para etiquetas o para mostrar información.

- **Text="{Binding LabelText, ElementName=root}"**: Esta es una expresión de enlace de datos (data binding).
 - **Binding LabelText**: Indica que el contenido de la propiedad Text de este TextBlock se enlazar  a una propiedad llamada LabelText.
 - **ElementName=root**: Especifica que la propiedad LabelText se buscar  en el elemento con x:Name="root", que es el UserControl padre. Esto significa que el UserControl tendr  una propiedad p blica llamada LabelText que puede ser establecida por el usuario del control, y el TextBlock mostrar  ese valor.
- **FontWeight="Bold"**: Establece el peso de la fuente del texto en negrita.

4. <TextBox Text="{Binding Text, ElementName=root, Mode=TwoWay}"/>

- **<TextBox>**: Es un control que permite al usuario introducir y editar texto.
- **Text="{Binding Text, ElementName=root, Mode=TwoWay}"**: Otra expresi n de enlace de datos.
 - **Binding Text**: Enlaza la propiedad Text de este TextBox a una propiedad llamada Text en el UserControl padre (root).
 - **ElementName=root**: De nuevo, indica que la propiedad Text se encuentra en el UserControl padre.
 - **Mode=TwoWay**: Esto es crucial. Significa que el enlace funciona en ambas direcciones:
 - Si la propiedad Text del UserControl cambia, el TextBox actualizar  su contenido.
 - Si el usuario escribe en el TextBox, el valor de la propiedad Text del UserControl se actualizar  autom ticamente. Esto permite que el UserControl exponga su valor de entrada de manera sencilla.

En resumen, este UserControl LabeledInput es un componente simple pero muy  til que consta de:

- Una etiqueta (TextBlock) en negrita, cuyo texto se define mediante una propiedad LabelText en el UserControl padre.
- Una caja de texto (TextBox) donde el usuario puede introducir texto, y cuyo valor se expone a trav s de una propiedad Text en el UserControl padre (con sincronizaci n bidireccional).

Este control permite que los desarrolladores usen un componente como este:

```
<Controls:LabeledInput LabelText="Nombre de Usuario:" Text="{Binding
UserNameProperty}"/>
```

En lugar de tener que escribir un TextBlock y un TextBox por separado y gestionar sus enlaces manualmente cada vez:

1. Antes de crear el componente

Si un desarrollador necesitara una etiqueta y una caja de texto, tendr a que escribir **varias l neas de c digo** cada vez:

```
XML
<StackPanel>
  <TextBlock Text="Nombre de Usuario:" FontWeight="Bold"/>
  <TextBox Text="{Binding PropiedadDelNombreDeUsuario}"/>
</StackPanel>
```

2. Despu s de crear el componente (LabeledInput)

Gracias a las dos "puertas" que usted est  creando (LabelText y Text), el desarrollador (su alumno) solo tendr  que escribir **una  nica l nea** para hacer exactamente lo mismo:

```
XML
<Controls:LabeledInput
  LabelText="Nombre de Usuario:" <--  Usa la Puerta 1!
  Text="{Binding UserNameProperty}" <--  Usa la Puerta 2!/>
```

Y ¿por qué al usar el componente habrá que poner un nuevo Binding?

- **Objetivo:** Conectar la **propiedad Text** del UserControl (la que usted creó) con una propiedad real de la aplicación, como UserNameProperty, que vive en el DataContext de la ventana.
- **Función:** Este segundo Binding toma el valor expuesto por el UserControl y lo pasa a la fuente de datos de la aplicación, cerrando el circuito.

Analogía del Puente (Resumen)

Imagina que el UserControl es un **Puente de Peaje** con dos extremos:

Extremo del Puente	¿Qué hace?
Lado Interno (nosotros lo programamos)	Se asegura de que el valor del TextBox llegue a la propiedad Text del componente (Puerta de Salida).
Lado Externo (El desarrollador lo usa)	Conecta la propiedad Text (Puerta de Salida) a la base de datos o al objeto de datos (UserNameProperty).

Si falta el Binding externo (Text="{Binding UserNameProperty}"), el valor que el usuario escribe en la caja de texto se guardará en la propiedad Text del componente, pero **nunca saldrá del componente** para llegar a la lógica de negocio de la aplicación.

3.2. LABELEDINPUT.XAML.CS

En el fichero labeled.xaml.cs copiamos el siguiente código:

```
using System.Windows;
using System.Windows.Controls;

namespace DemoControles.Controls
{
    public partial class LabeledInput : UserControl
    {
        public LabeledInput() => InitializeComponent();

        // Definimos las dos propiedades del control que estamos creando:

        // Propiedad LabelText
        public static readonly DependencyProperty LabelTextProperty =
            DependencyProperty.Register(nameof(LabelText), typeof(string),
                typeof(LabeledInput), new PropertyMetadata("Etiqueta"));

        public string LabelText
        {
            get => (string)GetValue(LabelTextProperty);
            set => SetValue(LabelTextProperty, value);
        }

        // Propiedad Text
        public static readonly DependencyProperty TextProperty =
            DependencyProperty.Register(nameof(Text), typeof(string),
                typeof(LabeledInput), new FrameworkPropertyMetadata("",
                    FrameworkPropertyMetadataOptions.BindsTwoWayByDefault));

        public string Text
        {
            get => (string)GetValue(TextProperty);
```

```

        set => SetValue(TextProperty, value);
    }
}
}

```

desglosamos a continuación la explicación del código:

Estructura General

- **using System.Windows; y using System.Windows.Controls;**: Estas directivas importan los espacios de nombres necesarios para trabajar con elementos de la interfaz de usuario de WPF, como UserControl y DependencyProperty.
- **namespace DemoControles.Controls;**: Define el espacio de nombres donde reside el control, organizando el código dentro del proyecto.
- **public partial class LabeledInput : UserControl**: Declara la clase LabeledInput que hereda de UserControl. La palabra clave partial indica que la definición de la clase se divide entre este archivo de código C# y un archivo XAML correspondiente (LabeledInput.xaml), donde se define la apariencia visual del control.
- **public LabeledInput() => InitializeComponent();**: Este es el constructor de la clase. Llama al método InitializeComponent(), que es generado automáticamente por WPF y se encarga de cargar el XAML asociado y construir la interfaz visual del control.

Propiedades de Dependencia (Dependency Properties)

Una de las características más potentes de WPF son las "Dependency Properties". Son un tipo especial de propiedad que amplía las propiedades estándar de C# para incluir funcionalidades de WPF como el enlace de datos (data binding), estilos, animaciones y herencia de valores.

1. Propiedad LabelText

- **public static readonly DependencyProperty LabelTextProperty**: Esta es la declaración de la propiedad de dependencia. Por convención, siempre se nombra añadiendo "Property" al final del nombre de la propiedad pública. Es estática y de solo lectura para asegurar que sea accesible a nivel de clase y no se modifique.
- **DependencyProperty.Register(...)**: Este método registra la propiedad de dependencia con el sistema de propiedades de WPF. Sus parámetros son:
 - nameof(LabelText): El nombre de la propiedad pública que envolverá a esta propiedad de dependencia.
 - typeof(string): El tipo de dato que almacenará la propiedad.
 - typeof(LabeledInput): El tipo de la clase propietaria de la propiedad.
 - new PropertyMetadata("Etiqueta"): Proporciona metadatos para la propiedad. En este caso, establece el valor por defecto de la propiedad a "Etiqueta".
- **public string LabelText { ... }**: Este es el "envoltorio" (wrapper) de CLR para la propiedad de dependencia. Proporciona el acceso get y set que se usaría normalmente en C#. Internamente, utiliza los métodos GetValue y SetValue para interactuar con el sistema de propiedades de WPF.

2. Propiedad Text

- **public static readonly DependencyProperty TextProperty**: De manera similar a LabelTextProperty, esta línea declara la propiedad de dependencia para el texto del campo de entrada.
- **DependencyProperty.Register(...)**: Registra la propiedad Text.

- `new FrameworkPropertyMetadata("", FrameworkPropertyMetadataOptions.BindsTwoWayByDefault)`: Aquí se utiliza `FrameworkPropertyMetadata`, una clase más específica que `PropertyMetadata`.
 - El primer parámetro "" establece el valor por defecto como una cadena vacía.
 - `FrameworkPropertyMetadataOptions.BindsTwoWayByDefault` es una configuración clave que indica que, por defecto, cualquier enlace de datos a esta propiedad será `TwoWay`. Esto significa que si el valor de la propiedad cambia en la interfaz de usuario (por ejemplo, el usuario escribe en un `TextBox`), el cambio se propagará automáticamente a la fuente de datos (como una propiedad de un `ViewModel`), y viceversa. Esta es una característica muy común y útil para los controles de entrada.

3.3. USO DEL COMPONENTE CREADO EN MAINWINDOW.XAML

Ahora vamos a usar este nuevo componente en la ventana principal del proyecto.

```
<Window ...
    xmlns:controls="clr-namespace:DemoControles.Controls">
y en el Grid
```

```
    <StackPanel Margin="20">
        <controls:LabeledInput LabelText="Nombre" Text="{Binding Nombre}" />
        <controls:LabeledInput LabelText="Email" Text="{Binding Email}" />
    </StackPanel>
</Window>
```

hemos invocado el control creado `LabeledInput`.

NOTA: no olvides compilar el proyecto!!

3.4. CREACION COMPONENTE CUSTOM CONTROL

3.4.1. Introducción Teórica: "La Piel y el Esqueleto"

- **User Control (.xaml + .cs):** Es como pegar varias piezas de Lego juntas. Es rápido, tiene interfaz fija y lógica pegada. Sirve para esa pantalla concreta.
- **Custom Control (.cs + Generic.xaml):** Es crear una pieza nueva de fábrica.
 - **El Esqueleto (.cs):** Define la lógica, las propiedades y los datos (C#).
 - **La Piel (XAML):** Define la apariencia visual (Template). Puede cambiarse totalmente sin tocar el código C#.

3.4.2. Conceptos Clave

1. **Herencia de Control:** No se hereda de `UserControl`, sino de `Control` (o `ContentControl` / `Button`, etc.).
2. **Dependency Properties (DP):** En WPF no se usan variables normales (`public int Valor {get;set;}`). Usamos DPs para que el control acepte **Binding** (`{Binding ...}`) y animaciones.
3. **Generic.xaml:** Es el archivo donde vive el estilo por defecto de nuestro control.

3.4.3. El Ejemplo Práctico: "NumberBox" (Selector Numérico)

Vamos a crear un control que tenga un botón de menos [-], un número y un botón de más [+].

Paso 1: La lógica (C#)

Creamos una clase simple (no tiene archivo XAML pegado):

```
using System.Windows;
using System.Windows.Controls;

namespace WpfApp.Controls
{
    public class NumberBox : Control
    {
        // 1. CONSTRUCTOR ESTÁTICO
        // Le dice a WPF: "Oye, busca mi estilo en Generic.xaml, no uses el
        // estilo por defecto"
        static NumberBox()
        {
            DefaultStyleKeyProperty.OverrideMetadata(typeof(NumberBox), new
            FrameworkPropertyMetadata(typeof(NumberBox)));
        }

        // 2. DEPENDENCY PROPERTY (El valor numérico)
        // Escribir "propdp" y pulsar TAB dos veces en Visual Studio crea
        // esto automáticamente.
        public int Value
        {
            get { return (int)GetValue(ValueProperty); }
            set { SetValue(ValueProperty, value); }
        }

        public static readonly DependencyProperty ValueProperty =
            DependencyProperty.Register("Value", typeof(int),
            typeof(NumberBox), new PropertyMetadata(0));

        // 3. LOGICA DE INTERACCIÓN
        // Sobreescribimos este método para buscar los botones en la
        // plantilla (Template)
        public override void OnApplyTemplate()
        {
            base.OnApplyTemplate();

            // Buscamos los botones por su nombre en el XAML (TemplatePart)
            Button btnUp = GetTemplateChild("PART_BtnUp") as Button;
            Button btnDown = GetTemplateChild("PART_BtnDown") as Button;

            // Ojo a las siguientes líneas de código, son delegados anónimos

            if (btnUp != null) btnUp.Click += (s, e) => Value++;
            if (btnDown != null) btnDown.Click += (s, e) => Value--;
        }
    }
}
```

Paso 2: La Apariencia (Generic.xaml)

Debe ir en una carpeta llamada Themes y dentro un archivo Generic.xaml. **WPF busca esta ruta automáticamente.**

```
<ResourceDictionary
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="clr-namespace:WpfApp.Controls">

    <!-- Estilo base para nuestro NumberBox -->
    <Style TargetType="{x:Type local:NumberBox}">
```

```

        <Setter Property="Template">
            <Setter.Value>
                <!-- Aquí definimos la estructura visual (ControlTemplate) -->
                <ControlTemplate TargetType="{x:Type local:NumberBox}">
                    <Border BorderBrush="Gray" BorderThickness="1" CornerRadius="5"
Background="White">
                        <StackPanel Orientation="Horizontal">

                            <!-- Botón Menos (Importante el nombre PART_) -->
                            <Button x:Name="PART_BtnDown" Content="-" Width="30"/>

                            <!-- El valor enlazado a la propiedad Value del control padre -->
                            <TextBlock Text="{TemplateBinding Value}"
                                VerticalAlignment="Center"
                                HorizontalAlignment="Center"
                                Width="50" TextAlignment="Center"
                                FontSize="16" FontWeight="Bold"/>

                            <!-- Botón Más -->
                            <Button x:Name="PART_BtnUp" Content="+" Width="30"/>

                        </StackPanel>
                    </Border>
                </ControlTemplate>
            </Setter.Value>
        </Setter>
    </Style>
</ResourceDictionary>

```

Paso 3: Cómo usarlo (MainWindow.xaml)

Ahora en la ventana principal usamos el control como si fuera un botón nativo de WPF.

```

<Window x:Class="WpfApp.MainWindow"
        xmlns:local="clr-namespace:WpfApp.Controls" ...>

    <StackPanel VerticalAlignment="Center" HorizontalAlignment="Center"
Spacing="20">

        <Label Content="Selecciona cantidad:"/>

        <!-- ¡Nuestro Custom Control! -->
        <local:NumberBox Value="5" Height="40"/>

        <!-- Otro ejemplo con Binding (suponiendo que tienes un ViewModel) --
    >
        <!-- <local:NumberBox Value="{Binding EdadUsuario}" /> -->

    </StackPanel>
</Window>

```

3.4.4. Puntos Didácticos Clave

1. Separación de Poderes:

- Pregunta: "¿Qué pasa si quiero que los botones sean redondos y rojos?"
- Respuesta: No tocas C#. Vas al Generic.xaml (o creas un nuevo Style en tu Window) y cambias el Template. La lógica de sumar/restar sigue funcionando intacta.

2. TemplateBinding:

- El {TemplateBinding Value} es un "túnel" que conecta lo que el usuario escribe en el XAML (<NumberBox Value="5">) con el TextBlock que está dentro de la plantilla.
3. **La convención PART_:**
- En OnApplyTemplate, buscamos elementos por nombre. Se usa el prefijo PART_ (convención estándar) para avisar a los diseñadores: *"Cuidado, si cambias el diseño, necesitas mantener un botón con este nombre o el código dejará de funcionar"*.

RESUMEN VISUAL

Concepto	User Control	Custom Control
Archivo	.xaml y .xaml.cs (Pareja)	.cs y Generic.xaml (Separados)
Uso	Pantallas específicas	Controles genéricos (Botones, Inputs)
Personalización	Difícil (Hardcoded)	Total (ControlTemplate)
Complejidad	Baja	Media/Alta

3.5. DISTRIBUCION DE COMPONENTES

Una vez que tienes un componente desarrollado queremos que sea útil y se pueda usar en otros proyectos. ¿Cómo podemos hacer esto? Fundamentalmente de dos maneras:

- **Referencia de Ensamblado (DLL):**
 - Compilas tu componente en una **Biblioteca de Clases WPF (WPF Class Library)**.
 - Tomas el archivo .dll resultante y lo agregas manualmente como referencia en el nuevo proyecto.
 - *Es el método "clásico" y rápido para pruebas locales.*
- **Paquete NuGet:**
 - Empaquetar la biblioteca en un archivo .nupkg.
 - Puedes alojarlo en una carpeta local, en un servidor privado o en nuget.org.
 - *Es el método "profesional", ya que maneja versiones (v1.0, v1.1) y dependencias automáticamente.*

3.5.1. Referencias de Ensamblado(DLL)

Vamos a crear un par de componentes muy simples de los dos tipos vistos anteriormente. Crearemos a continuación una librería que usaremos en otro proyecto.

PASO 1: CREAR EL PROYECTO "CONTENEDOR" (LA BIBLIOTECA)

No se crea una "WPF App". Debe crearse una biblioteca:

1. Abrir Visual Studio.
2. Crear un nuevo proyecto.
3. Buscar la plantilla: **WPF Class Library** (o BIBLIOTECA DE CLASES DE WPF).
 - NOTA: Asegúrate de elegir la versión correcta (.NET Framework o .NET 6/8) según lo que usarán tus futuros proyectos.
4. Lo llamamos MiLibreriaWPF (ojo a mayúsculas y minúsculas)

Una vez creado, borrar el Class1.cs que viene por defecto.

PASO 2: AÑADIR UN USERCONTROL

Vamos a crear un control simple (ej. una tarjeta de información).

1. Clic derecho en el proyecto MiLibreriaWPF -> **Agregar -> Nuevo Elemento -> Control de usuario (WPF)**.
2. Llámalo TarjetaInfo.xaml.
3. **El código (XAML):** Ponle un borde y un texto para reconocerlo.

```
<UserControl x:Class="MiLibreriaWPF.TarjetaInfo"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Height="100" Width="200">
    <Border Background="LightBlue" CornerRadius="10" BorderBrush="Navy"
        BorderThickness="2">
        <TextBlock Text="Soy un UserControl (DLL)"
            HorizontalAlignment="Center" VerticalAlignment="Center"
            FontWeight="Bold"/>
    </Border>
</UserControl>
```

PASO 3: AÑADIR UN CUSTOMCONTRO

Ahora vamos a crear un botón personalizado.

1. Clic derecho en el proyecto MiLibreriaWPF -> **Agregar -> Nuevo Elemento -> Control personalizado (WPF)**.
2. Llámalo BotonRedondo.cs.
3. Visual Studio creará:
 - La clase BotonRedondo.cs.
 - Una carpeta Themes con un archivo Generic.xaml (Aquí es donde vive el diseño de los CustomControls).

- A. La Clase (BotonRedondo.cs):** Solo asegúrate de que tenga el constructor estático que le dice a WPF "busca mi estilo en Generic.xaml".

```
using System.Windows;
using System.Windows.Controls;

namespace MiLibreriaWPF
{
    public class BotonRedondo : Button // Heredamos de Button
    {
        static BotonRedondo()
        {
            // Esta línea es VITAL. Sin ella, el control no se verá.
            DefaultStyleKeyProperty.OverrideMetadata(typeof(BotonRedondo),
new FrameworkPropertyMetadata(typeof(BotonRedondo)));
        }
    }
}
```

- B. El Estilo (Themes/Generic.xaml):** Define cómo se ve. Busca el <Style> que se generó y modifícalo:

```

        <ResourceDictionary
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:local="clr-namespace:MiLibreriaWPF">

    <Style TargetType="{x:Type local:BotonRedondo}">
        <Setter Property="Template">
            <Setter.Value>
                <ControlTemplate TargetType="{x:Type local:BotonRedondo}">

                    <Grid>
                        <Ellipse Fill="Red" Stroke="Black"/>
                        <ContentPresenter HorizontalAlignment="Center"
VerticalAlignment="Center"/>
                    </Grid>
                </ControlTemplate>
            </Setter.Value>
        </Setter>
    </Style>
</ResourceDictionary>

```

PASO 4: MAPEO DE XMLNS

Para que quien use la DLL no tenga que escribir rutas largas en XAML, vamos a darle un nombre más simple.

1. Abre el archivo AssemblyInfo.cs (En .NET Framework está en la carpeta Properties. En .NET 6/8 modernos, simplemente crea un archivo nuevo llamado AssemblyInfo.cs en la raíz del proyecto).
2. Añade esto al final:

```

using System.Windows.Markup;

// Esto permite que en el otro proyecto usen
xmlns:miscontroles="http://miscontroles.com"
[assembly: XmlnsDefinition("http://miscontroles.com", "MiLibreriaWPF")]

```

PASO 5: GENERAR LA DLL

1. Ve al menú **Compilar -> Compilar Solución**.
2. Ve a la carpeta de tu proyecto en Windows: MiLibreriaWPF\bin\Debug\netX.X\ (o Release).
3. Ahí verás **MiLibreriaWPF.dll**. ¡Ese es tu producto final!

PASO 6: CÓMO USARLA EN OTRO PROYECTO

Ahora imagina que abres un proyecto nuevo (MiAppFinal).

1. En el "Explorador de Soluciones", haz clic derecho en **Dependencias** (o Referencias) -> **Agregar Referencia de proyecto -> COM->Examinar**.
2. Dale a **Examinar (Browse)** y busca tu archivo .dll generado en el paso 5.
3. Abre tu MainWindow.xaml y añade el espacio de nombres:

Opción A (Si hiciste el Paso 4 - Recomendado):

```
<Window ...  
    xmlns:mis="http://miscontroles.com"> <!-- Mucho más limpio -->  
  
    <StackPanel>  
        <mis:TarjetaInfo Margin="10"/>  
        <mis:BotonRedondo Content="Click" Width="50" Height="50"/>  
    </StackPanel>  
</Window>
```

Opción B (Sin el Paso 4 - Método Clásico): Tendrías que declarar el namespace apuntando al ensamblado:

```
xmlns:mis="clr-namespace:MiLibreriaWPF;assembly=MiLibreriaWPF"
```